

Reconfigurable Newton Hardware

DFG-to-ASIC Architecture, Unified Mathematical PE Fabric, Hardware Gradient/Hessian Generation, Matrix Solve and FSM Flow

Final project boundary

Software performs only the one-time conversion of a supported nonlinear expression into a primitive-operation Data Flow Graph (DFG). After loading, the ASIC autonomously evaluates the equation, generates gradient and Hessian values, solves the Newton correction matrix, updates variables and checks convergence.

The physical datapath remains unchanged for different supported equations. Only the DFG program, constants, variable count and numerical controls change.

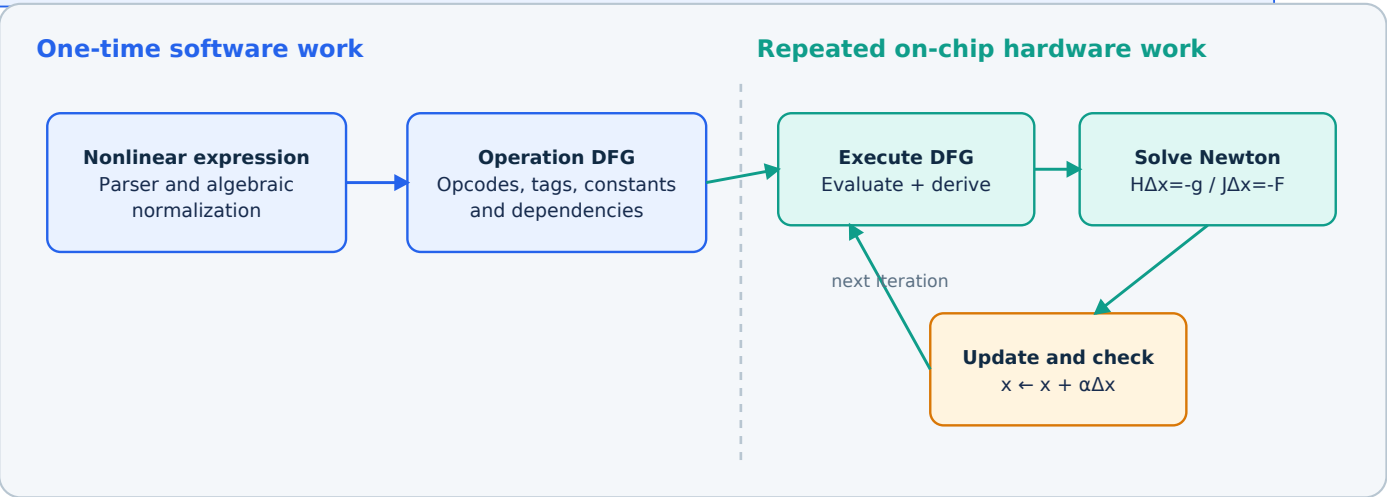


Figure 1. Software stops at the DFG boundary; the complete repeated Newton loop is on chip.

Terminology used in this note

Term	Meaning in this architecture
DFG	Data Flow Graph: primitive operation nodes connected by data dependencies.
Linear operation representation	An ordered/dependency-tagged ADD/SUB/MUL/DIV program. It is not the final numerical Newton matrix.
Newton linear system	The hardware-generated correction $H \Delta x = -g$ at the current iteration.
Generic	The same silicon executes supported smooth equations expressible by the opcode set, derivative rules and configured size/precision limits. This is not a claim to solve every mathematical equation.

1. Equation DFG: the only software-generated object

A DFG node contains an opcode, source tags, destination tag, constants or immediate references, and dependency/ready information. The DFG preserves the original mathematical result while exposing independent nodes that hardware may execute in parallel.

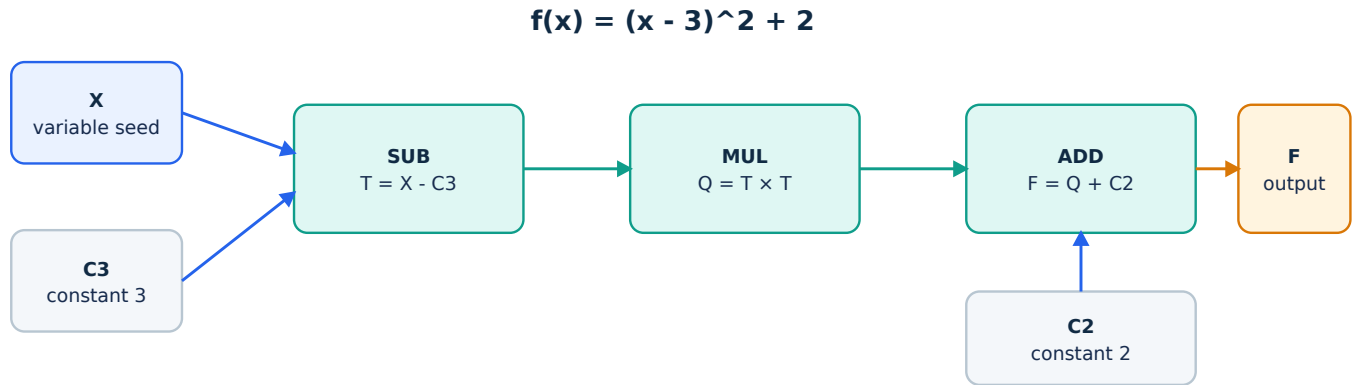


Figure 2. Exact primitive-operation DFG for $f(x) = (x - 3)^2 + 2$.

Node	Mathematics	Operation record	Dependency
N0	$t = x - 3$	SUB T, X, C3	X and C3 ready
N1	$q = t \times t$	MUL Q, T, T	Wait for N0
N2	$f = q + 2$	ADD F, Q, C2	Wait for N1

Two-variable example

For $f(x,y) = x^2 + xy + 2y^2 - 3x - 5y$, software emits only the following operation program. It does not emit numerical gradient or Hessian values.

```

MUL T1,X,X | MUL T2,X,Y | MUL T3,Y,Y | MUL T4,C2,T3
MUL T5,C3,X | MUL T6,C5,Y | ADD T7,T1,T2 | ADD T8,T7,T4
SUB T9,T8,T5 | SUB F,T9,T6 | END
  
```

Parallelism exposed by the DFG

Dependency level	Ready operations	Possible hardware execution
0	$T1=x \times x, T2=x \times y, T3=y \times y, T5=3x, T6=5y$	Dispatch across available RMPes; remaining nodes wait.
1	$T4=2T3, T7=T1+T2$	MUL and ADD may execute concurrently.
2	$T8=T7+T4$	Wait for both source tags.
3-4	$T9=T8-T5; F=T9-T6$	Final dependent subtraction chain.

Hardware-ready instruction fields: mode | opcode | srcA | srcB | optional srcC | dst | variable/tile indices | dependency tags | final flag.

At run time, the on-chip scoreboard—not software—decides which ready DFG nodes issue to available operator resources. Depending on derivative mode, an edge carries either a sampled scalar value or an augmented value/derivative record.

2. What the mentor's 'matrix format' can mean

Three different structures must not be mixed. The operation-scheduling matrix represents the equation program; the Newton matrix is numerical and is generated at run time; the PE/tile matrix is the physical hardware organization.

Matrix meaning	Rows / columns	When produced	Purpose
Operation schedule	Rows=dependency levels; columns=operator banks or PE slots	Software tags dependencies; hardware selects ready issue slots	Expose legal parallelism without changing arithmetic order.
Newton numerical matrix	Hessian H for optimization, or Jacobian J for a root system	Generated numerically in hardware at every iteration	Solve $H\Delta x = -g$ or $J\Delta x = -F$.
Physical PE/tile array	Rows/columns of reused RMPes, memories and routes	Fixed in silicon	Execute DFG, derivative and matrix-solve operations.

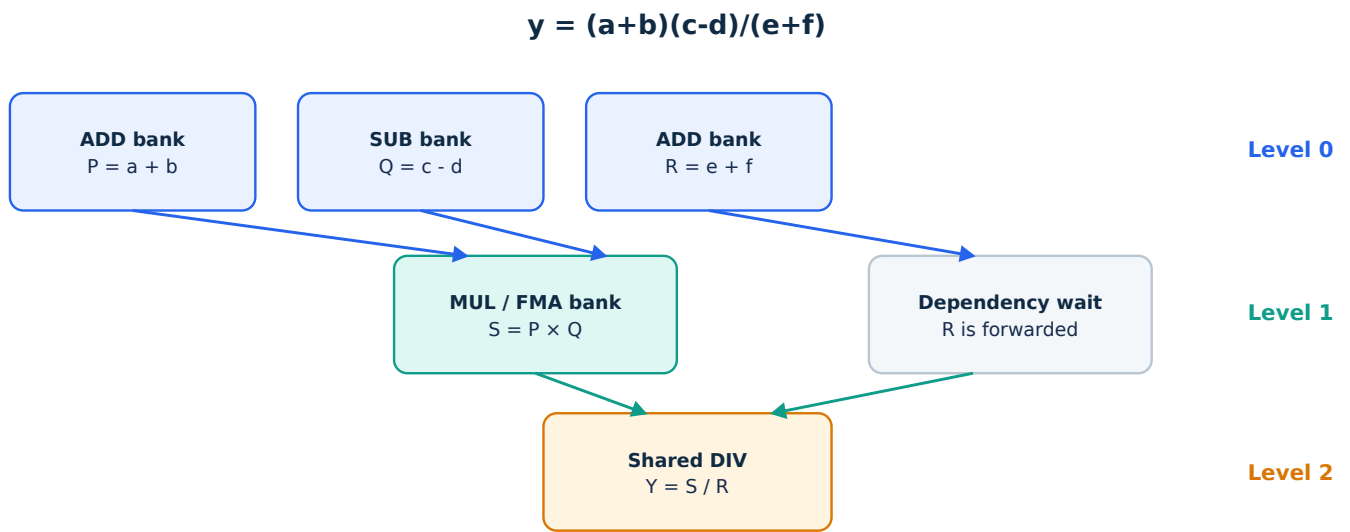


Figure 3. A dependency-level schedule: three independent operations issue together; multiplication and division wait for their source tags.

Level	ADD/SUB bank	MUL/FMA bank	DIV bank	Ready after level
0	$P=a+b; Q=c-d; R=e+f$	—	—	P,Q,R
1	—	$S=P \times Q$	—	S,R
2	—	—	$Y=S/R$	Y

This schedule is a matrix-like representation of primitive operations; it does not claim that a nonlinear function has already become one numerical linear equation. The actual Newton linear system appears only after hardware evaluates the derivatives at the current x .

3. Unified hardware: one arithmetic fabric, many modes

The evaluator, derivative engine, matrix solver and update controller issue work to one shared reconfigurable fabric. This arithmetic reuse is the key small-block idea: different equations change instructions and data, not the silicon datapath.

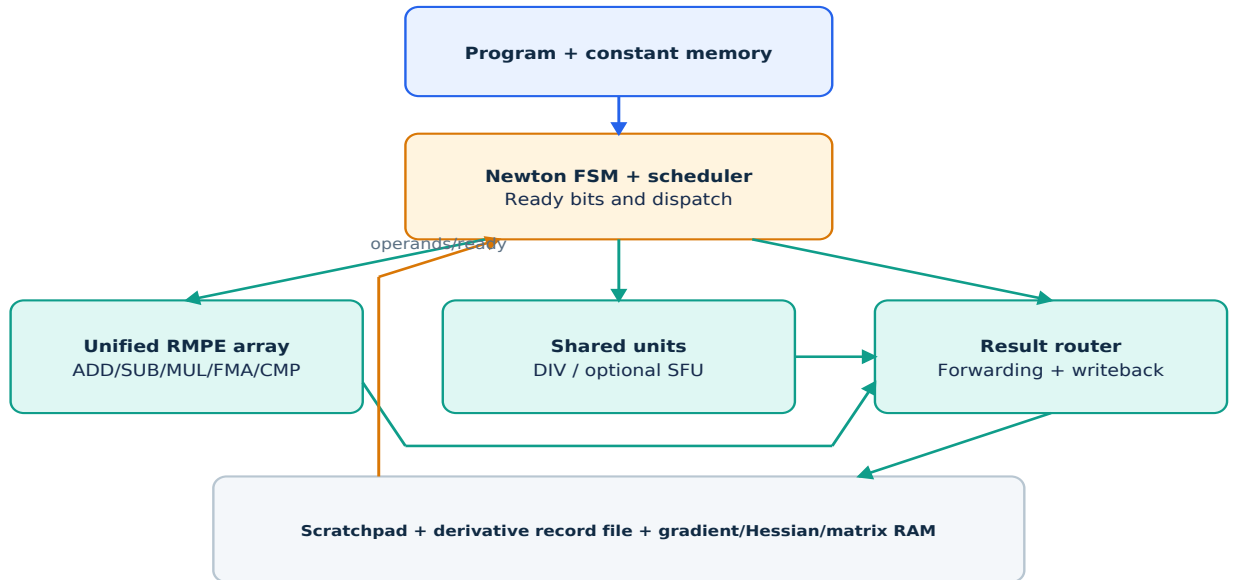


Figure 4. Hardware flow graph. The on-chip scheduler dispatches ready DFG nodes and the same fabric executes the complete Newton loop.

The generic small block: RMPE

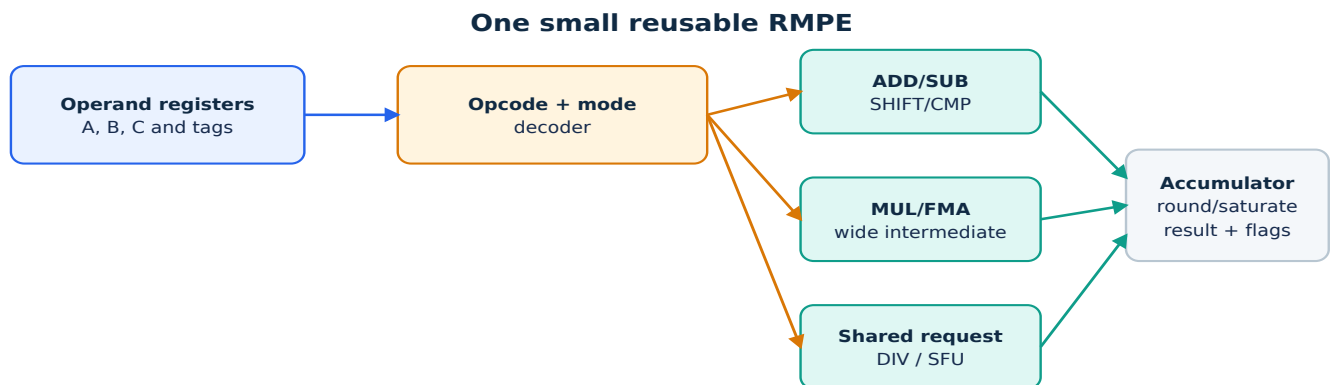


Figure 5. Reconfigurable Mathematical Processing Element (RMPE). Expensive DIV/SFU resources are shared.

Mode	Work mapped to the same fabric
VALUE / SAMPLE	Original DFG evaluation or finite-difference sample evaluation.
GRAD / HESS	Derivative-rule propagation or difference-formula accumulation.
SOLVE	Factorization, matrix updates, dot products and substitutions.
UPDATE / CHECK	FMA variable update; ABS/CMP, bounds, denominator and convergence checks.

Recommended first fabric: two parameterized RMPEs, one shared iterative divider, local scratchpad, derivative/matrix RAM, result router, ready-node scheduler and bounded Newton FSM.

4. Two hardware ways to obtain gradient and Hessian

The earlier architecture mixed two choices. They should be treated as a baseline and an optimization candidate, because they change storage, control and numerical behavior.

Baseline: central finite differences, all calculations on chip

$f_{-}=f(x-h), f_0=f(x), f_{+}=f(x+h)$		
$g \approx (f_{+}-f_{-})/(2h), H \approx (f_{+}-2f_0+f_{-})/h^2$		
Property	Finite-difference baseline	Forward second-order AD
Input program	Ordinary value DFG	Same DFG plus derivative-aware opcode rules
Hardware action	Evaluate shifted samples, then subtract/multiply/divide	Propagate value, gradient and Hessian records
Accuracy	Approximate; sensitive to h and fixed-point cancellation	Exact derivative of supported opcode implementation, subject to arithmetic rounding
1D work	Three function evaluations per iteration	One augmented DFG traversal
n-D dense work	About $2n^2+1$ function samples for full central Hessian	One traversal but $O(n^2)$ derivative data/update work
ASIC complexity	Simpler evaluator; more repeated cycles	More derivative RAM, routing and derivative-rule control

Recommended research sequence

Stage	Deliverable	Reason
A	Implement scalar finite difference using the generic evaluator and shared divider.	Lowest-risk proof that all Newton numerical work is in hardware.
B	Implement scalar second-order AD using augmented records.	Demonstrates the faster derivative-aware architecture.
C	Compare cycles, area, power, error and convergence across both modes.	Creates the optimization result expected from research.
D	Extend the chosen path to n=2; tile gradient/Hessian storage for larger n.	Avoid an unbounded 'any equation/any size' RTL target.

Mentor confirmation needed: whether finite difference is the required first implementation or only the reference baseline. The PDF therefore specifies both without silently replacing one with AD.

5. Derivative-aware DFG mathematics

For scalar second-order AD, each live value is $R=(v,d1,d2)$. Seed $X=(x,1,0)$ and constants $C=(c,0,0)$. The same DFG becomes derivative-aware through fixed opcode rules.

ADD/SUB: $v=a\pm b$, $d1=a'\pm b'$, $d2=a''\pm b''$

MUL: $v=ab$, $d1=a'b+ab'$, $d2=a''b+2a'b'+ab''$

DIV is implemented as reciprocal plus multiply

$r=1/b$, $r'=-b'/b^2$, $r''=2(b')^2/b^3-b''/b^2$; $u=a\cdot r$ uses the MUL rule

The divider must reject $|b|$ below a programmable threshold and report DIV_BY_ZERO or trigger a bounded damping/regularization retry. Special functions require a defined LUT, polynomial or CORDIC implementation and matching derivative rules.

DFG step	Input record(s)	Output at $x=0$ for $f=(x-3)^2+2$
Seed	$X=(0,1,0)$, $C3=(3,0,0)$, $C2=(2,0,0)$	Inputs ready
$T=X-C3$	SUB rule	$T=(-3,1,0)$
$Q=T\times T$	MUL rule	$Q=(9,-6,2)$
$F=Q+C2$	ADD rule	$F=(11,-6,2)$

Hardware directly produces $f(0)=11$, $g=-6$ and $H=2$. Software provides only X , constants and the DFG records.

Multivariable extension

For $u=ab$: $g_u=b g_a+a g_\beta$; $H_u=bH_a+aH_\beta+g_a g_\beta^T+g_\beta g_a^T$. Variable x_i is seeded with basis vector e_i and a zero Hessian. Store gradient components and symmetric Hessian tiles in RAM instead of widening every RMPE by n^2 .

Equation feature	Hardware support condition
Polynomial/rational	Supported by ADD/SUB/MUL/DIV rules, with denominator guards.
exp/log/sin/cos	Only if an SFU implementation and consistent first/second derivative rules exist.
abs/max/branch	Piecewise evaluation is possible, but derivative behavior at boundaries must be specified.
Discontinuity	A comparator can evaluate it, but ordinary Newton derivatives may be invalid; return status or use another method.

6. Newton modes and on-chip matrix solve

Mode	Hardware-generated data	Linear system solved	Update
Optimization	Objective f, gradient g, Hessian H	$(H+\lambda I)\Delta x = -g$	$x \leftarrow x + \alpha \Delta x$
Nonlinear root system	Residual vector F and Jacobian J	$J\Delta x = -F$	$x \leftarrow x + \alpha \Delta x$

Both modes use the same DFG evaluator, arithmetic fabric, matrix RAM, divider and solve controller. They differ in the loaded output tags, derivative target and controller program.

Optimization example

For $f(x,y)=x^2+xy+2y^2-3x-5y$ at $(0,0)$: $g=[-3,-5]^T$, $H=[[2,1],[1,4]]$, $H\Delta x=[3,5]^T$

A 2x2 LDLT sequence avoids explicit inversion

Step	Mathematics	Hardware issue	Result
1	$d1=H11$; $l21=H21/d1$	MOV + shared DIV	2; 0.5
2	$d2=H22-l21H21$	RMPE FMA	3.5
3	$z2=b2-l21z1$	RMPE FMA	3.5
4	$w1=z1/d1$; $w2=z2/d2$	Shared DIV	1.5; 1
5	$\Delta y=w2$; $\Delta x=w1-l21\Delta y$	RMPE SUB/FMA	1; 1

Matrix solve reuses the same arithmetic fabric

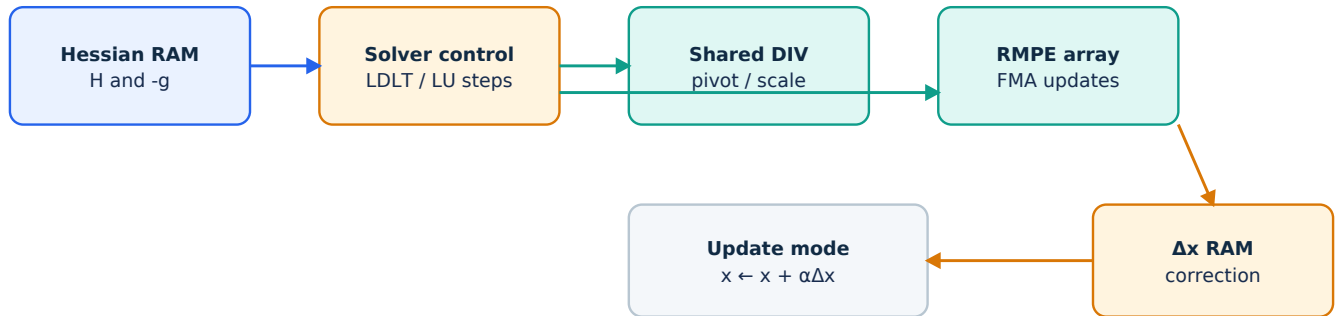


Figure 6. Matrix solving reuses the RMPEs and divider; only the solve controller changes execution mode.

Case	Preferred bounded solve path
$n=1$	Guarded scalar division.
SPD Hessian	Cholesky or LDLT without forming H^{-1} .
Indefinite/general matrix	Pivoted LDLT or pivoted LU fallback.
Weak/singular Hessian	Try bounded regularization $H+\lambda I$; otherwise return SINGULAR_MATRIX/BAD_HESSIAN.

7. Corrected hardware FSM flow

The previous flow returned a non-converged iterate directly to DFG execution. That is incorrect because iteration-local ready bits, derivative seeds and accumulators must be reinitialized. The corrected flow also makes step rejection and regularization bounded.

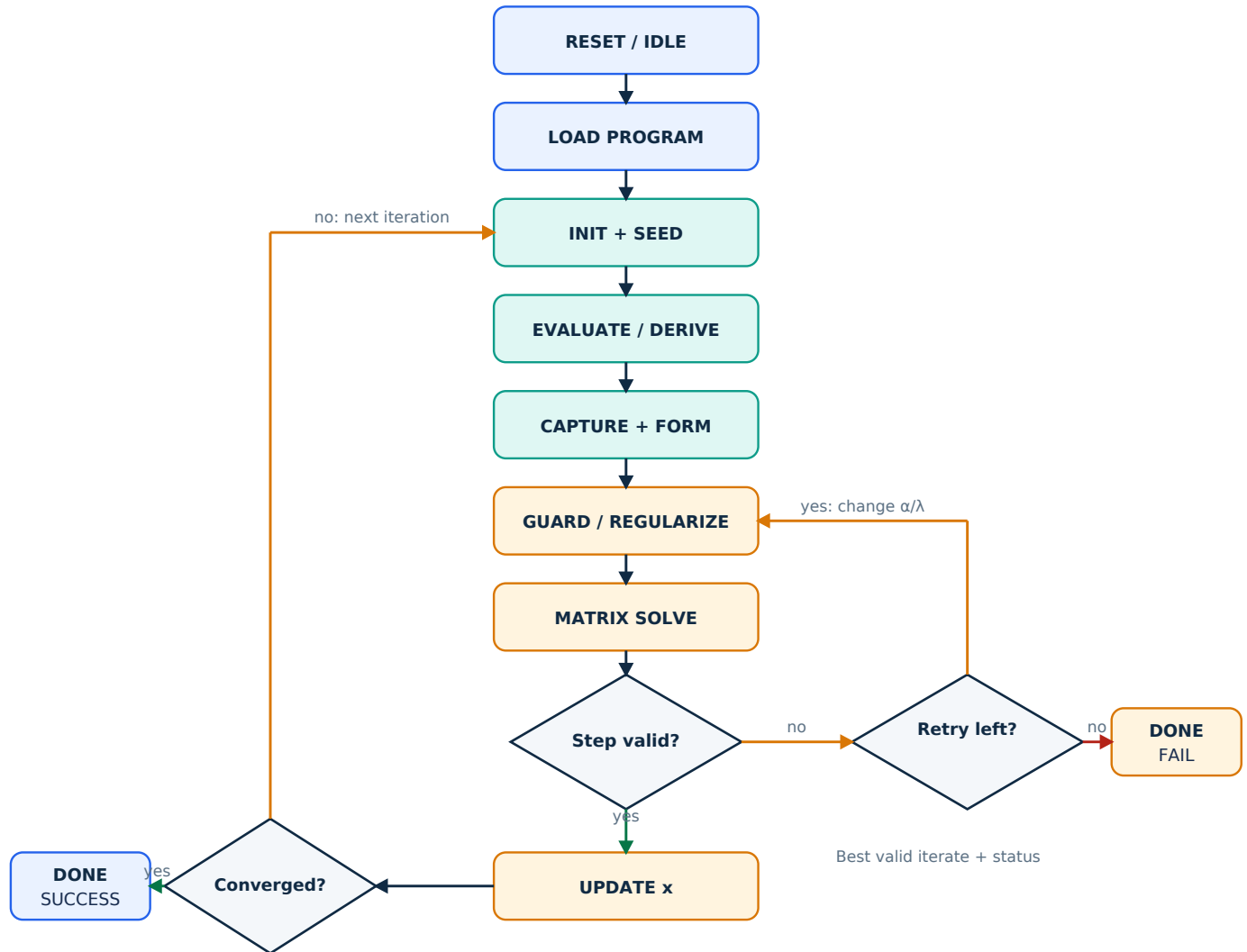


Figure 7. Corrected Newton FSM: every new iteration passes through INIT+SEED; rejected steps retry α/λ only within a fixed bound.

State group	Main action	Exit
LOAD	Validate opcodes, sizes, addresses, constants and controls.	START or PROGRAM_ERROR
INIT + SEED	Clear iteration state; seed samples or derivative records.	All input tags ready
EVALUATE / DERIVE	Scan ready nodes, issue, wait, write back; repeat for samples/tiles.	Output tags complete
CAPTURE + FORM	Store $f/g/H$ or F/J ; write A and b matrix RAM.	System ready
GUARD / REGULARIZE	Check overflow, denominator/pivot and add λI if configured.	Safe solve or bounded failure
MATRIX SOLVE	Scalar/LDLT/Cholesky/LU controller reuses RMPE and DIV.	Candidate Δx or solver error
CHECK STEP / RETRY	Test candidate; reduce α or raise λ on rejection.	Accept, retry, or STEP_REJECTED
UPDATE / CONVERGENCE	Update x; test residual/gradient/step and iteration bound.	DONE or NEXT_ITERATION
DONE	Return best valid iterate, count, flags and status.	Host acknowledge/reset

8. ASIC module organization and bounded first target

```
reconfigurable_newton_accelerator
├─ config_interface + program/constant memory
├─ newton_fsm + ready_node_scheduler + scoreboard
├─ unified_rmpe_fabric
│ └─ rmpe[0 : NUM_RMPE-1]
│   └─ shared_divider / optional_sfu
├─ result_router + value/derivative scratchpad
├─ gradient / Hessian / Jacobian / matrix RAM
├─ derivative_or_sample_controller
├─ matrix_solve_controller
├─ update_guard_convergence_controller
└─ result_status_interface
```

Recommended first bounded target

Parameter	Initial value	Reason
Number format	Signed Q16.16	Bit-accurate starting point; use wide intermediates and explicit rounding/saturation.
Derivative mode	Finite difference first; AD selectable next	Preserves the confirmed baseline and creates a measurable optimization.
Variables	n=1, then n=2	Verify derivative and matrix paths before tiled scaling.
RMPE / divider	2 RMPEs / 1 shared iterative DIV	Parallel ready-node issue with controlled area.
Program/register depth	64 / 64 entries, parameterized	Bounded demonstrator; not an unlimited-equation claim.
Matrix solver	Scalar DIV, then 2x2 LDLT	No explicit inverse; clear cycle-by-cycle verification.

Implementation sequence

Phase	Verified deliverable
0	Freeze opcode/DFG record, fixed-point policy, memory map, latencies and supported scope.
1	RMPE arithmetic, flags, shared divider and self-checking unit tests.
2	Program memory, scratchpad, scoreboard, result router and ready-node issue.
3	Value DFG evaluator; prove multiple equations on unchanged RTL.
4	Finite-difference g/H path and scalar Newton FSM, including guards and bounded retries.
5	Second-order AD path; compare with Phase 4 in cycles/area/power/error.
6	n=2 storage, 2x2 matrix solve and optimization/root-mode controller tests.
7	Synthesis, timing, utilization, throughput and numerical-convergence report.

Required statuses

SUCCESS | MAX_ITER | BAD_HESSIAN | SINGULAR_MATRIX | DIV_BY_ZERO | OVERFLOW | STEP_REJECTED | PROGRAM_ERROR | UNSUPPORTED_OP | TIMEOUT

9. Alignment with the mentor and next review

Items already aligned

Mentor expectation	Architecture response
Do not build one circuit per equation	One fixed RMPE/divider/memory/control fabric executes loaded DFG programs.
Split and channelize operations	Dependency-tagged nodes issue to ADD/SUB, MUL/FMA, DIV/SFU and memory resources.
Find independence and dependency	Scoreboard and ready-node scheduler perform on-chip issue decisions.
Hardware computes Newton work	Evaluator, g/H or F/J generation, matrix solve, update and convergence are on chip.
Use a matrix/tensor-style organization	Operation-level schedule, numerical Newton matrix and physical PE/tile array are explicitly separated.
Optimize after correctness	A bounded scalar baseline precedes AD, $n=2$, parallelism and synthesis comparisons.

Questions to confirm with the mentor before RTL

Decision	Recommended proposal
Meaning of matrix format	Present all three interpretations on page 3 and ask which one is the primary research representation.
Derivative baseline	Implement hardware finite difference first; add second-order AD as the optimized comparison.
Scheduling	Keep readiness/dispatch in hardware; software supplies dependency tags only.
Newton problem class	Support optimization first; retain $J\Delta x = -F$ as a configured extension.
First limits	Q16.16, $n=1$, 64 instructions, two RMPEs and one divider; then $n=2$.
Throughput target	Define equations/second or cycles/iteration before choosing more RMPEs or matrix tiles.

Acceptance demonstrations

1) Load $f(x)=(x-3)^2+2$. At $x=0$, hardware produces $f=11$, $g=-6$, $H=2$, $\Delta x=3$ and converges to $x=3$. 2) Without RTL changes, load the two-variable quadratic; hardware produces $g=[-3,-5]^T$, $H=[[2,1],[1,4]]$, solves $\Delta x=[1,1]^T$ and updates $x=[1,1]^T$.

Scope statement for the research report

The accelerator is equation-generic within a declared numerical domain: smooth functions that can be represented by the supported DFG opcode/derivative library, bounded program and variable sizes, and selected fixed-point range. Equations outside that domain must return UNSUPPORTED_OP or be approximated by an explicitly verified SFU implementation.

Next action

Review pages 3, 5 and 8 with the mentor. After those three decisions are approved, freeze the Phase 0 microarchitecture specification before generating RTL.